The great thing about Q-learning is that it learns directly from the available information in the environment, which are the rewards. It basically learns to predict rewards, which we call values. If we use a DQN to play pinball, it will learn to predict the values for the two main actions—operating the left and right paddles. We're then free to use these values to decide which action to take, generally opting for the action associated with the highest value.

A policy gradient function is more directly connected to the concept of *reinforcement*, since we positively reinforce actions that result in a positive reward and negatively reinforce actions that lead to a negative reward. Hence, the policy function learns which actions are best in a more hidden way. In pinball, if we hit the left paddle and score a bunch of points, that action will get positively reinforced and will be more likely to be selected the next time the game is in a similar state.

In other words, Q-learning (such as DQN) uses a trainable function to directly model the value (the expected reward) of an action, given a state. This is a very intuitive way of solving a Markov decision process (MDP) since we only observe states and rewards—it makes sense to predict the rewards and then just take actions that have high predicted rewards. On the other hand, we saw the advantage of direct policy learning (such as policy gradients). Namely, we get a true conditional probability distribution over actions, $P(a|S)$, that we can directly sample from to take an action. Naturally, someone decided it might be a good idea to combine these two approaches to get the advantages of both.

In building such a combined value-policy learning algorithm, we will start with the policy learner as the foundation. There are two challenges we want to overcome to increase the robustness of the policy learner:

- We want to improve the sample efficiency by updating more frequently.
- We want to decrease the variance of the reward we used to update our model.